Implementation Spec — Document Upload & Background Ingestion

Goal: a user uploads a PDF, gets an immediate response, and ingestion (chunking → embedding → question generation) runs in the background. The frontend polls until the quiz is ready.

Stack: FastAPI, PostgreSQL + pgvector, existing `llm/` modules (`chunking.py`, `generation.py`, `rag/embedding.py`).

---

1. Why it's built this way

Ingestion takes 60–120 seconds for a small document, and several minutes for a large one. Browsers and reverse proxies time out HTTP requests at roughly 30–60 seconds, so doing this work inside the request handler would simply fail.

The solution splits it into two phases:

Phase A — the request (fast, ~100ms). Validate the file, write it to disk, insert a `documents` row with `status='pending'`, register a background task, return the `document_id`.

Phase B — the background task (slow, minutes). Reopen the file from disk, run the full pipeline, update `status` as it progresses, write chunks and questions to the database.

The `documents.status` column is the only communication channel between the two phases. The frontend polls a status endpoint until it reads `ready`.

---

2. Files to create or modify

```
app/
├── routes/
│   └── rag_routes.py          # MODIFY — add the two endpoints
├── controllers/
│   └── document_controller.py # CREATE — file handling + DB writes
├── services/
│   └── ingestion_service.py   # CREATE — the background pipeline
├── schemas/
│   └── document_schemas.py    # CREATE — response models
└── db/
    └── document_queries.py    # CREATE — SQL for documents/chunks/questions
```

The reason for this split: `ingestion_service.py` is **not triggered by HTTP**. It runs as a background task and needs to read and update documents independently. If that logic lived in the route file, the service would have to import from a routes module, which tangles the dependency graph. Anything that would be identical if this were a CLI command belongs outside the route.

---

3. Response schemas

File: `app/schemas/document_schemas.py`

python

```python
from typing import Optional
from datetime import datetime
from pydantic import BaseModel


class NewChatResponse(BaseModel):
    document_id: int
    filename: str
    status: str          # always "pending" at this point


class DocumentStatusResponse(BaseModel):
    document_id: int
    filename: str
    status: str          # pending | processing | ready | failed
    error: Optional[str] = None
    chunk_count: Optional[int] = None
    question_count: Optional[int] = None
    created_at: datetime
    processed_at: Optional[datetime] = None
```

Note: there is no *request* schema for the upload. Pydantic parses JSON bodies; a file arrives as `multipart/form-data`, which FastAPI handles with `UploadFile` directly.

`chunk_count` and `question_count` are included in the status response so the frontend can show meaningful progress ("47 questions ready") rather than just a spinner.

---

4. The route

File: `app/routes/rag_routes.py`

python

```python
import os
from fastapi import (
    APIRouter, UploadFile, File, Depends,
    BackgroundTasks, HTTPException, status,
)

from app.schemas.document_schemas import NewChatResponse, DocumentStatusRespons
from app.controllers import document_controller
from app.services.ingestion_service import run_ingestion
from app.auth.dependencies import get_current_user

router = APIRouter(prefix="/rag", tags=["rag"])

MAX_UPLOAD_BYTES = 20 * 1024 * 1024          # 20 MB


@router.post(
    "/new-chat",
    response_model=NewChatResponse,
    status_code=status.HTTP_201_CREATED,
)
async def new_chat(
    background_tasks: BackgroundTasks,
    file: UploadFile = File(...),
    current_user = Depends(get_current_user),
):
    """Upload a PDF and start ingestion in the background."""

    # --- validation ----------------------------------------------------
    if not file.filename or not file.filename.lower().endswith(".pdf"):
        raise HTTPException(400, "Only PDF files are supported")

    if file.content_type not in ("application/pdf", "application/octet-stream"):
        raise HTTPException(400, "Invalid file type")

    contents = await file.read()

    if len(contents) == 0:
        raise HTTPException(400, "Uploaded file is empty")

    if len(contents) > MAX_UPLOAD_BYTES:
        raise HTTPException(413, "File too large (max 20 MB)")

    if not contents.startswith(b"%PDF"):
        raise HTTPException(400, "File does not appear to be a valid PDF")
```

```
    # --- save + create row ------------------------------------------
    doc = document_controller.create_document(
        user_id=current_user.user_id,
        original_filename=file.filename,
        contents=contents,
    )

    # --- queue the slow work ----------------------------------------
    background_tasks.add_task(run_ingestion, doc["document_id"])

    return doc


@router.get("/documents/{document_id}", response_model=DocumentStatusResponse)
async def get_document_status(
    document_id: int,
    current_user = Depends(get_current_user),
):
    """Poll this until status is 'ready' or 'failed'."""
    doc = document_controller.get_document_for_user(
        document_id=document_id,
        user_id=current_user.user_id,
    )
    if doc is None:
        raise HTTPException(404, "Document not found")
    return doc
```

Why each validation exists

Check	Reason
`.pdf` extension	Cheapest first filter
`content_type`	Second signal; some clients send `octet-stream` for PDFs, so both are accepted
Empty file	An empty file would fail confusingly deep inside PyMuPDF
Size cap	Prevents a 500-page book from consuming the entire LLM budget in one upload
`%PDF` magic bytes	The only check that can't be spoofed by renaming a `.docx` to `.pdf`

The ownership check is not optional

`get_document_for_user` filters on **both** `document_id` and `user_id`. Because the schema uses integer primary keys, `/rag/documents/47` is trivially guessable — a user can try 46, 45, 44 and read other people's documents unless every endpoint verifies ownership. This applies to every endpoint that touches a document or session.

---

5. The controller

File: `app/controllers/document_controller.py`

python

```python
import os
import uuid

from app.db import document_queries

UPLOAD_DIR = os.getenv("UPLOAD_DIR", "uploads")


def create_document(user_id: int, original_filename: str, contents: bytes) -> d
    """Write the PDF to disk and insert its documents row."""
    os.makedirs(UPLOAD_DIR, exist_ok=True)

    # storage_path is generated by us, never derived from user input
    storage_path = os.path.join(UPLOAD_DIR, f"{uuid.uuid4()}.pdf")

    with open(storage_path, "wb") as f:
        f.write(contents)

    return document_queries.insert_document(
        user_id=user_id,
        filename=original_filename,
        storage_path=storage_path,
        status="pending",
    )


def get_document_for_user(document_id: int, user_id: int) -> dict | None:
    return document_queries.select_document_for_user(document_id, user_id)
```

The two-filename rule

Every uploaded file has two names, and they must never be confused:

Column	Value	Used for
`filename`	`"Compiler Unit 1.pdf"`	**Display only.** Shown in the sidebar.
`storage_path`	`"uploads/a3f2c81e-9d4b-....pdf"`	**Opening the file only.** Never shown.

Why generate a UUID instead of using the user's filename:

1. **Collisions.** Two students both upload `unit1.pdf`; the second would overwrite the first.
2. **Path traversal.** A filename like `../../etc/passwd` would write outside the upload directory. User-supplied strings must never appear in a filesystem path.

Why UUID rather than `{document_id}.pdf`: using the id would require inserting the row first to obtain it, then building the path, then updating the row — three operations. A UUID is known before the insert, so one insert suffices.

---

6. The database layer

File: `app/db/document_queries.py`

python

```python
from datetime import datetime, timezone
from app.db.connection import get_connection


def insert_document(user_id: int, filename: str,
                    storage_path: str, status: str) -> dict:
    sql = """
        insert into documents (user_id, filename, storage_path, status)
        values (%s, %s, %s, %s)
        returning document_id, filename, status
    """
    with get_connection() as conn, conn.cursor() as cur:
        cur.execute(sql, (user_id, filename, storage_path, status))
        row = cur.fetchone()
        conn.commit()
    return {"document_id": row[0], "filename": row[1], "status": row[2]}


def select_document_for_user(document_id: int, user_id: int) -> dict | None:
    """Includes counts so the frontend can show real progress."""
    sql = """
        select d.document_id, d.filename, d.status, d.error,
               d.created_at, d.processed_at,
               (select count(*) from chunks    c where c.document_id = d.docume
               (select count(*) from questions q where q.document_id = d.docume
        from documents d
        where d.document_id = %s and d.user_id = %s
    """
    with get_connection() as conn, conn.cursor() as cur:
        cur.execute(sql, (document_id, user_id))
        row = cur.fetchone()

    if row is None:
        return None

    return {
        "document_id":    row[0],
        "filename":       row[1],
        "status":         row[2],
        "error":          row[3],
        "created_at":     row[4],
        "processed_at":   row[5],
        "chunk_count":    row[6],
        "question_count": row[7],
    }
```

```
def set_document_status(document_id: int, status: str,
                        error: str | None = None,
                        processed: bool = False) -> None:
    sql = """
        update documents
        set status = %s,
            error = %s,
            processed_at = case when %s then now() else processed_at end
        where document_id = %s
    """
    with get_connection() as conn, conn.cursor() as cur:
        cur.execute(sql, (status, error, processed, document_id))
        conn.commit()


def get_storage_path(document_id: int) -> str | None:
    with get_connection() as conn, conn.cursor() as cur:
        cur.execute(
            "select storage_path from documents where document_id = %s",
            (document_id,),
        )
        row = cur.fetchone()
    return row[0] if row else None
```

Bulk insert for chunks

Chunks must be inserted with `execute_values`, not in a loop — a 100-page document produces hundreds of rows, and one round trip per row is unnecessarily slow.

python

```
from psycopg2.extras import execute_values


def insert_chunks(document_id: int, chunks: list[dict]) -> list[int]:
    """Returns chunk_ids in the same order as the input list."""
    sql = """
        insert into chunks
            (document_id, idx, content, topic, parent,
             page_from, page_to, embedding)
        values %s
        returning chunk_id
    """
    values = [
        (
            document_id,
            i,
            c["content"],
            c["topic"],
            c.get("parent"),
            c.get("page_from"),
            c.get("page_to"),
            c.get("embedding"),
        )
        for i, c in enumerate(chunks)
    ]

    with get_connection() as conn, conn.cursor() as cur:
        rows = execute_values(cur, sql, values, fetch=True)
        conn.commit()
    return [r[0] for r in rows]


def insert_questions(document_id: int, questions: list[dict]) -> None:
    import json

    sql = """
        insert into questions
            (document_id, chunk_id, question_text, ideal_answer,
             key_points, topic, parent, difficulty)
        values %s
    """
    values = [
        (
            document_id,
            None,                               # see note below
            q["question"],
```

```
            q["ideal_answer"],
            json.dumps(q["key_points"]),       # jsonb column
            q["topic"],
            q.get("parent"),
            q["difficulty"],
        )
        for q in questions
    ]

    with get_connection() as conn, conn.cursor() as cur:
        execute_values(cur, sql, values)
        conn.commit()
```

Why `chunk_id` is `None` on questions

Questions are generated **per topic**, not per chunk. A single topic can span several chunks — either because a long topic was split for size, or because the document covers it in two places. There is therefore no single chunk a question "came from," so `chunk_id` is nullable and left empty.

When grading needs the source material for a question, it fetches by topic instead:

sql

```
select string_agg(content, E'\n\n' order by idx)
from chunks
where document_id = %s and topic = %s
```

This makes topic-string consistency critical. `chunks.topic` and `questions.topic` must match exactly, or that join silently returns nothing and grading loses its grounding with no error raised. Topic strings are normalized once during chunking — trailing colons stripped, whitespace collapsed — and the normalized form is written to both tables. Never normalize at query time.

Note on `key_points`

It's a `jsonb` column, so the Python list must be `json.dumps`'d on insert. If you later switch to SQLAlchemy or asyncpg, that serialization is handled automatically and the explicit dumps must be removed.

---

7. The ingestion service (the background task)

File: `app/services/ingestion_service.py`

python

```python
import asyncio
import logging

import fitz  # PyMuPDF

from app.db import document_queries
from llm.ingestion.chunking import chunk_document
from llm.ingestion.generation import QuestionGenerator
from llm.rag.embedding import Embedding

log = logging.getLogger(__name__)


async def run_ingestion(document_id: int) -> None:
    """
    Full pipeline. Runs as a FastAPI background task, so it must never
    raise — any exception is caught and recorded on the document row.
    """
    try:
        document_queries.set_document_status(document_id, "processing")

        # --- 1. read the PDF ----------------------------------------
        storage_path = document_queries.get_storage_path(document_id)
        if not storage_path:
            raise ValueError("Document row is missing its storage_path")

        pdf = fitz.open(storage_path)
        toc = pdf.get_toc()

        full = ""
        page_starts = []
        for p in range(pdf.page_count):
            page_starts.append(len(full))
            full += pdf[p].get_text()

        # --- 2. guard: scanned / image-only PDFs ----------------------
        if len(full.strip()) < 200:
            raise ValueError(
                "No extractable text found. This PDF appears to be scanned "
                "or image-based; please upload a text-based PDF."
            )

        # --- 3. chunk -------------------------------------------------
        chunks = chunk_document(full, page_starts, toc)
        if not chunks:
            raise ValueError("Chunking produced no sections from this document"
```

```
        log.info("doc %s: %d chunks", document_id, len(chunks))

        # --- 4. embed + generate, concurrently ------------------------
        # Independent of each other: both read chunk content, neither
        # reads the other's output. Embedding takes seconds; generation
        # takes minutes. Running them together means the embedding time
        # disappears inside the generation time.
        embedder = Embedding(chunks)
        generator = QuestionGenerator(chunks)

        embedded_chunks, questions = await asyncio.gather(
            embedder.generateEmbedding(),
            generator.generateQuestions(),
        )

        # --- 5. persist -----------------------------------------------
        # Chunks first: they must exist before anything references them,
        # and retrieval is useless without them.
        document_queries.insert_chunks(document_id, embedded_chunks)

        if questions:
            document_queries.insert_questions(document_id, questions)
        else:
            raise ValueError("Question generation produced no questions")

        log.info("doc %s: %d questions", document_id, len(questions))

        # --- 6. done --------------------------------------------------
        document_queries.set_document_status(
            document_id, "ready", processed=True
        )

    except Exception as e:
        log.exception("ingestion failed for document %s", document_id)
        document_queries.set_document_status(
            document_id, "failed", error=str(e)[:500]
        )
```

Points worth understanding in this function

It must never raise. A background task has no HTTP client waiting for a response, so an uncaught exception disappears into the server logs and the document is left in `processing` forever. The blanket `except` is deliberate, and it writes the reason into `documents.error` so the user sees something actionable.

The scanned-PDF guard is the failure you will actually hit. A scanned or photographed document has no text layer, so `get_text()` returns almost nothing. Without this check, chunking produces zero sections and the failure surfaces much deeper with a confusing message. Handwritten notes are explicitly out of scope for v1, and this is the check that enforces it cleanly.

Embedding and generation run concurrently because they are independent. Both read `chunk["content"]`; neither reads the other's output. Since embedding takes seconds and generation takes minutes, `asyncio.gather` makes the embedding cost effectively zero.

Chunks are inserted before questions. Even though `questions.chunk_id` is null, chunks must exist for retrieval to work at all, and the grading path fetches source material from them.

`error` **is truncated to 500 characters.** LLM exception messages can be enormous — the `tool_use_failed` errors from Groq include the entire failed generation payload, which can run to thousands of characters. Storing that untruncated bloats the row for no benefit.

---

8. Registering the router

File: `app/main.py`

```python
from fastapi import FastAPI
from app.routes import auth_routes, rag_routes

app = FastAPI(title="Voice AI Study Coach")

app.include_router(auth_routes.router)
app.include_router(rag_routes.router)
```

---

9. The full request flow

```
POST /rag/new-chat   (multipart, file=unit1.pdf)
    ↓
validate: extension, content-type, non-empty, size, %PDF magic bytes
    ↓
write bytes to uploads/{uuid}.pdf
    ↓
insert documents row → status='pending'
    ↓
background_tasks.add_task(run_ingestion, document_id)
    ↓
RESPONSE 201  { document_id: 5, filename: "unit1.pdf", status: "pending" }
    ↓   (~100 ms — the user is no longer waiting)
    |
    └── background task begins
            status → 'processing'
            open PDF, extract text, check for text layer
            chunk_document()                    ~10 s
            embed + generate concurrently       ~60–120 s
            insert chunks, insert questions
            status → 'ready', processed_at = now()

Meanwhile the frontend polls:
    GET /rag/documents/5  →  { status: "processing", chunk_count: 0,
question_count: 0 }
    GET /rag/documents/5  →  { status: "processing", chunk_count: 42,
question_count: 0 }
    GET /rag/documents/5  →  { status: "ready",      chunk_count: 42,
question_count: 310 }
                              ↑ enable the "Start quiz" button here
```

Poll every 2–3 seconds. `chunk_count` and `question_count` let the UI show real progress instead of an indeterminate spinner.

---

10. Known limitation and its mitigation

FastAPI background tasks run **inside the web server process**. If the server restarts mid-ingestion, the task is lost and the document remains stuck in `processing` indefinitely.

This is acceptable for v1, but it needs a cleanup job:

```
python
```

```
def reap_stuck_documents():
    sql = """
        update documents
        set status = 'failed',
            error  = 'Ingestion did not complete (server restarted)'
        where status = 'processing'
          and created_at < now() - interval '30 minutes'
    """
```

Run it on application startup and, optionally, on a periodic timer.

If this becomes a recurring problem in practice, that is the point at which Celery + Redis earns its place — a real task queue survives restarts and gives you retries. Do not reach for it before then; it is significant additional infrastructure for a problem that a 30-minute reaper handles adequately at this stage.

---

11. Deferred: starting the quiz before generation finishes

The design includes a future enhancement where the quiz becomes available as soon as the first few topics have questions, while the rest continue generating in the background. The student answers roughly one question per 45 seconds; generation produces a topic every few seconds, so the generator stays comfortably ahead.

That would add a fifth status value:

```sql
status text check (status in ('pending','processing','partial','ready','failed'
```

`partial` means "usable but still filling in" — the frontend enables the start button on `partial` or `ready`.

Do not build this yet. Get the straightforward version working end to end first. The change is contained: one new status value, one flip to `partial` after the first few topics complete, and a short wait-and-retry in question selection for the rare case where a student outruns the generator.

---

12. Checklist

- [ ] `app/schemas/document_schemas.py` — `NewChatResponse`, `DocumentStatusResponse`
- [ ] `app/db/document_queries.py` — insert/select/update documents, bulk insert chunks and questions
- [ ] `app/controllers/document_controller.py` — UUID path generation, file write, DB insert
- [ ] `app/services/ingestion_service.py` — `run_ingestion`, wrapped so it cannot raise
- [ ] `app/routes/rag_routes.py` — `POST /rag/new-chat`, `GET /rag/documents/{id}`

- [ ] Register the router in `main.py`
- [ ] `UPLOAD_DIR` in `.env`, and added to `.gitignore`
- [ ] Startup reaper for documents stuck in `processing`
- [ ] Verify ownership filtering on **every** endpoint that takes a `document_id`